# Guion de Defensa EP4 y Banco de Preguntas

## Evaluación Parcial N.º 4 · ISY1102

**Sección 002V**

Carrera: Ingeniería en Desarrollo de Software

Docente: Christian Rodrigo Alvarez Lobos

Fecha: 17 de octubre de 2026

**Integrantes:**

Luis Tasso

Miguel Arredondo

Nicolas Iturrieta

## 1. Estructura y distribución del tiempo

La defensa tiene 10 minutos de exposición y 5 minutos de preguntas. La defensa es individual aunque el trabajo sea grupal: cada integrante debe dominar el contenido completo.

| N.º | Diapositiva | Inicio | Fin | Duración |
|---|---|---|---|---|
| 1 | Portada y saludo | 0:00 | 0:30 | 30 s |
| 2 | Contenido | 0:30 | 0:50 | 20 s |
| 3 | Contexto y alcance | 0:50 | 1:40 | 50 s |
| 4 | Enfoque estratégico | 1:40 | 2:40 | 60 s |
| 5 | Tipos de prueba | 2:40 | 3:20 | 40 s |
| 6 | Herramientas | 3:20 | 4:10 | 50 s |
| 7 | Resumen de ejecución | 4:10 | 5:00 | 50 s |
| 8 | Casos representativos | 5:00 | 5:50 | 50 s |
| 9 | Análisis estático | 5:50 | 6:40 | 50 s |
| 10 | Clasificación de defectos | 6:40 | 7:30 | 50 s |
| 11 | Evidencias | 7:30 | 8:00 | 30 s |
| 12 | Reflexiones | 8:00 | 9:00 | 60 s |
| 13 | Dictamen y acciones | 9:00 | 9:40 | 40 s |
| 14 | Cierre | 9:40 | 10:00 | 20 s |

## 2. Guion diapositiva por diapositiva

### Diapositiva 1 · Portada (0:00–0:30)

**Mensaje clave:** presentar al equipo, el caso y adelantar el dictamen.

> *Buenos días. Somos Luis Tasso, Miguel Arredondo y Nicolas Iturrieta. Presentamos la verificación de seguridad y calidad del sistema Atlas, desarrollada para la Evaluación Parcial N.º 3. Anticipo la conclusión: el dictamen es NO CERTIFICADO, porque encontramos cuatro defectos críticos de seguridad que impiden su puesta en producción. A continuación explico el enfoque, la ejecución y la evidencia.*

### Diapositiva 2 · Contenido (0:30–0:50)

**Mensaje clave:** ordenar la exposición.

> *La presentación sigue el orden exigido: contexto y alcance, enfoque estratégico de las pruebas, tipos y herramientas, resultados de ejecución, análisis estático, clasificación de los*

*defectos, reflexiones de calidad, seguridad y normativa, y finalmente el dictamen con las acciones correctivas.*

### Diapositiva 3 · Contexto y alcance (0:50–1:40)

**Mensaje clave:** qué es Atlas, qué módulos se evaluaron y con qué marco.

*Atlas es una plataforma web para pymes chilenas que gestiona clientes, contratos y documentos. Tiene arquitectura de tres capas: frontend React con Material UI, API REST en Express/Node y PostgreSQL. Evaluamos seis módulos: autenticación, clientes, contratos y documentos, usuarios y roles, métricas y auditoría.*

>

*Nos guiamos por ISO/IEC 25010 para calidad, ISO/IEC 27001 y OWASP Top 10 para seguridad, y la Ley 19.628 y 21.719 de protección de datos. El objetivo era determinar con evidencia si el software es apto para producción. Todo se ejecutó sobre un despliegue Docker en laboratorio: frontend en el puerto 3350, API en 4450 y PostgreSQL en 15432.*

### Diapositiva 4 · Enfoque estratégico (1:40–2:40)

**Mensaje clave:** cómo se planificó y priorizó el ciclo de pruebas.

*Aplicamos un enfoque mixto: ejecución manual reproducible con curl, análisis estático automatizado y revisión manual del código. La priorización fue por riesgo: primero seguridad —control de acceso, subida de archivos e inyección—, luego la funcionalidad central y al final lo no funcional.*

>

*Trabajamos en dos ciclos: el Ciclo 1 fue un smoke test de funciones base; si fallaba, deteníamos la ejecución. El Ciclo 2 ejecutó 12 casos funcionales y de seguridad más pruebas complementarias. Los criterios de entrada exigían entorno desplegado, datos semilla y casos revisados; los de salida, el 100 % de los casos críticos ejecutados, sin defectos críticos abiertos y con evidencia registrada.*

### Diapositiva 5 · Tipos de prueba (2:40–3:20)

**Mensaje clave:** cada tipo responde a un requisito del ERS.

*Diseñamos cinco tipos de prueba, cada uno justificado por el ERS. Funcionales para validar los requisitos RF-1 a RF-6: login, CRUD de clientes, contratos, usuarios, dashboard y*

*auditoría. De seguridad para control de acceso, uploads, SQLi, CORS y fuerza bruta. De integración para verificar el flujo completo entre frontend, API y base de datos. De rendimiento para comparar los tiempos de respuesta con los umbrales NFR-PERF. Y de usabilidad, en una revisión acotada de los estados de interfaz.*

## Diapositiva 6 · Herramientas (3:20–4:10)

**Mensaje clave:** justificar la elección de cada herramienta.

*Elegimos cada herramienta por su aporte a la evidencia. Docker Compose reproduce el ambiente completo de forma aislada y verificable. curl nos permite controlar token, cabeceras y payloads, y registrar el código HTTP como prueba. ESLint detecta errores y malas prácticas de calidad de forma automática. SonarQube entrega métricas objetivas de bugs, vulnerabilidades, hotspots y duplicación. Y la revisión manual cubre lo que las herramientas no ven: control de acceso, secretos y manejo de archivos. El equipo de prueba fue un líder de QA, un tester funcional y un tester de seguridad, sobre un entorno QA local con datos ficticios.*

## Diapositiva 7 · Resumen de ejecución (4:10–5:00)

**Mensaje clave:** resultados cuantitativos del ciclo.

*Diseñamos y ejecutamos 12 casos: 10 aprobados, un 83 % de éxito, y 2 fallidos. Los fallos son CP-010, que aceptó un archivo ejecutable en la subida de documentos, y CP-011, que permitió leer datos de contratos sin token. Sumamos 10 pruebas de seguridad complementarias, de las cuales 7 arrojaron resultado inseguro. En total registramos 11 incidentes: 4 críticos, 5 graves, 1 moderado y 1 leve. ESLint reportó 14 problemas y SonarQube, 1 vulnerabilidad BLOCKER, 4 hotspots y 198 code smells. El tiempo de respuesta fue de 4 segundos por una latencia artificial en el código.*

## Diapositiva 8 · Casos representativos (5:00–5:50)

**Mensaje clave:** conectar cada caso con su evidencia.

*Estos son los casos que mejor resumen el resultado. CP-001 aprobó: el login válido emite el JWT. CP-010 falló: el sistema aceptó un archivo .exe y lo sirvió como PDF, lo que es un riesgo de malware. CP-011 falló: un GET a contratos sin token devolvió los datos completos, un IDOR clásico. CP-012 sí aprobó: el intento de inyección SQL fue rechazado con 401 porque las consultas están parametrizadas. SEG-12 mostró CORS abierto con Allow-Origin asterisco, y FN-02 midió 4,06 segundos frente a un umbral de 300 milisegundos.*

## Diapositiva 9 · Análisis estático (5:50–6:40)

**Mensaje clave:** la herramienta confirma los hallazgos dinámicos.

*El análisis estático confirma lo que vimos dinámicamente. ESLint detectó 14 problemas, principalmente variables e imports sin uso y dependencias de hooks. SonarQube, en el backend, encontró una vulnerabilidad BLOCKER: la contraseña de PostgreSQL escrita directamente en db.js, línea 6. Además, 4 security hotspots y un Security Rating E. En el frontend encontró 1 bug, 181 code smells y un problema crítico de complejidad cognitiva. Lo relevante es que los hotspots coinciden exactamente con los defectos BUG-003, de subida de archivos, y BUG-005, de CORS.*

## Diapositiva 10 · Clasificación de defectos (6:40–7:30)

**Mensaje clave:** escala aplicada y regla de certificación.

*Clasificamos los 11 incidentes con una escala de cuatro niveles. CRITICO: 4 casos, que impiden operar o exponen datos sensibles —el acceso sin token, las credenciales en texto claro, la subida sin validación y el secreto en el código—. GRAVE: 5 casos, entre ellos el JWT_SECRET inseguro, el CORS abierto, la ausencia de rate-limit, los errores 500 y el uso de localStorage. MODERADO: la latencia artificial. Y LEVE: los code smells de ESLint. La regla que aplicamos es que un software con defectos CRITICO abiertos nunca debe certificarse.*

## Diapositiva 11 · Evidencias (7:30–8:00)

**Mensaje clave:** mostrar que todo es verificable.

*Aquí están las capturas del servidor SonarQube local. Se ven los dashboards de ambos proyectos, los issues de code smells y del frontend, el Quality Gate y el resumen de proyectos. Todas las capturas están incorporadas también en el Anexo E del informe, para que la evidencia sea verificable.*

## Diapositiva 12 · Reflexiones (8:00–9:00)

**Mensaje clave:** calidad, seguridad, normativa, legal y ética.

*En calidad, el sistema tiene una base funcional correcta, pero una deuda técnica alta en el frontend y un bug de compatibilidad con React 19. En seguridad, los cuatro defectos críticos se mapean a OWASP: control de acceso roto, fallas criptográficas, fallas de autenticación y de integridad de datos y software.*

>

*En lo normativo, la exposición de datos personales y el manejo de contraseñas incumplen la Ley 19.628 y la 21.719, y los controles de acceso y criptografía de ISO 27001. En lo legal, los datos de clientes no pueden quedar accesibles sin autorización. Y en lo ético, nuestro compromiso fue reportar todo con transparencia y evidencia, sin ocultar hallazgos y sin usar datos reales.*

### Diapositiva 13 · Dictamen (9:00–9:40)

**Mensaje clave:** criterio, resultado y acciones.

*El criterio de certificación era tener el 100 % de los casos de seguridad críticos exitosos y ningún defecto crítico abierto. No se cumplió: fallaron CP-010 y CP-011 y quedan 4 incidentes CRITICO. Por eso el dictamen es NO CERTIFICADO. Las acciones correctivas prioritarias son: autenticación global en todos los endpoints, hash seguro de contraseñas, validación de archivos, secretos fuera del código, CORS restringido, rate-limit, transacciones y errores genéricos, cookies httpOnly y un Quality Gate de seguridad en CI.*

### Diapositiva 14 · Cierre (9:40–10:00)

**Mensaje clave:** cerrar y ofrecer respaldo.

*Con esto cerramos la presentación. El informe completo, con el detalle de cada caso y las evidencias, está disponible como respaldo. Quedamos atentos a sus preguntas.*

## 3. Preguntas probables del docente y respuestas modelo

### 3.1. Sobre metodología y enfoque

**P: ¿Por qué el dictamen es NO CERTIFICADO si el 83 % de los casos pasó?**

R: Porque el criterio de certificación no es el promedio, sino dos condiciones obligatorias: el 100 % de los casos de seguridad críticos exitosos y ningún defecto de severidad crítica abierto. Fallaron CP-010 y CP-011, ambos de seguridad, y quedan 4 incidentes CRITICO. Un software con defectos críticos abiertos nunca debe certificarse.

**P: ¿Cómo se calculó la cobertura del 100 % del ERS?**

R: Con una matriz de trazabilidad bidireccional: los 12 requisitos evaluados (RQ-01 a RQ-12) se vincularon a 12 casos de prueba (CP-001 a CP-012), cada uno con resultado y defecto asociado. La matriz está en la sección 4.2 del informe.

**P: ¿Cuál es la diferencia entre prueba funcional, de seguridad y no funcional en su plan?**

R: La funcional verifica qué hace el sistema (login, CRUD, contratos). La de seguridad verifica que no pueda hacerse lo que no corresponde (acceso sin token, uploads, inyección). La no funcional mide cómo lo hace (rendimiento, usabilidad). Cada una responde a requisitos distintos del ERS.

**P: ¿Por qué usaron dos ciclos y no uno solo?**

R: El Ciclo 1 es un smoke test: si las funciones base fallan, no tiene sentido ejecutar el resto. El Ciclo 2 profundiza con los 12 casos y las pruebas complementarias. Es una práctica estándar de ISTQB para optimizar tiempo y aislar fallos.

**P: ¿La defensa es individual; cómo se distribuyó el trabajo?**

R: El trabajo de ejecución se dividió por especialidad —funcional, seguridad y análisis estático—, pero todos revisamos el informe completo y el dictamen. Por eso cualquiera de nosotros puede explicar cualquier sección.

### 3.2. Sobre herramientas y análisis estático

**P: ¿Por qué usaron SonarQube si ya tenían ESLint?**

R: Porque miden cosas distintas. ESLint es un linter de JavaScript: detecta errores de estilo, variables sin uso y dependencias de hooks. SonarQube analiza seguridad, hotspots, duplicación, complejidad y entrega ratings. De hecho, solo SonarQube detectó la vulnerabilidad BLOCKER de credenciales en db.js y los hotspots de CORS y upload.

**P: ¿Por qué el Quality Gate aparece OK si hay una vulnerabilidad BLOCKER?**

R: Porque el Quality Gate por defecto ("Sonar way") no tenía condiciones de seguridad activas para el proyecto: evalúa principalmente bugs y deuda técnica sobre código nuevo. Las métricas sí reflejan el problema: Security Rating E. Por eso una de nuestras recomendaciones es configurar un Quality Gate con condiciones de seguridad en CI.

**P: ¿Qué significa un Security Rating E?**

R: Es el peor nivel de la escala (5.0). Indica que existe al menos una vulnerabilidad de severidad alta o bloqueante sin resolver. En el backend corresponde exactamente a la contraseña de PostgreSQL en db.js:6.

**P: ¿Qué diferencia hay entre un security hotspot y una vulnerabilidad?**

R: La vulnerabilidad es un problema confirmado que debe corregirse. El hotspot es un punto que requiere revisión humana: SonarQube lo marca como potencialmente riesgoso y el analista decide. Nuestros 4 hotspots coincidieron con defectos reales que ya habíamos confirmado en pruebas dinámicas.

**P: ¿El análisis estático reemplaza las pruebas dinámicas?**

R: No, son complementarias. El análisis estático no detecta lógica de autorización por empresa ni flujos de negocio; las pruebas dinámicas sí. Y las pruebas dinámicas pueden no ver problemas de configuración o deuda técnica; el estático sí. Usamos ambos.

**P: ¿Por qué no usaron OWASP ZAP o JMeter, que estaban en el plan del EP1?**

R: Por alcance y tiempo del encargo. Priorizamos pruebas dirigidas de control de acceso y análisis estático, que atacaban los riesgos identificados en el EP1. ZAP y JMeter quedan como trabajo futuro para un ciclo de pentest y carga completo; lo declaramos como limitación.

### 3.3. Sobre hallazgos técnicos

**P: ¿Qué es un IDOR y cómo lo evidenciaron?**

R: Insecure Direct Object Reference: se accede a un objeto por su identificador sin validar autorización. Lo evidenciamos con `GET /contratos/1` sin cabecera Authorization: el servidor respondió 200 con los datos completos. Lo mismo ocurrió con usuarios, auditoría y roles (SEG-01, SEG-03, SEG-04, SEG-05).

**P: ¿El problema es falta de autenticación o de autorización?**

R: Ambos. Falta un middleware global de autenticación —los endpoints no validan token— y además no se valida la pertenencia a la empresa ni el rol, que es

autorización. El control de roles está delegado al frontend, por lo que se puede omitir llamando la API directamente.

**P: ¿Cómo probaron la inyección SQL y por qué no funcionó?**

R: Enviamos el payload `admin' OR '1'='1` en el login. El servidor respondió 401 porque todas las consultas usan parámetros `$1..$n` de node-postgres, que separan datos de instrucciones. Es el caso CP-012, aprobado.

**P: ¿Por qué aceptar un archivo .exe es crítico?**

R: Porque no hay filtro de tipo MIME ni de extensión en multer: se aceptó el ejecutable y además se sirve como `application/pdf`. Eso permite almacenar y distribuir malware desde la propia plataforma. Se confirmó con SEG-07b, SEG-08b y SEG-11a.

**P: ¿Qué riesgo genera un CORS con asterisco?**

R: Permite que cualquier origen haga peticiones a la API. Combinado con credenciales, facilita CSRF y exfiltración de datos. Debe restringirse a los orígenes autorizados (BUG-005, hotspot de SonarQube).

**P: ¿Qué implica que el JWT_SECRET sea "inseguro"?**

R: Si el secreto es conocido o por defecto, un atacante puede firmar tokens válidos y suplantar usuarios. Está replicado en siete routers, así que el impacto es global sobre la autenticación (BUG-004).

**P: ¿Por qué las credenciales en texto claro son un hallazgo legal y no solo técnico?**

R: Porque la Ley 19.628 y la 21.719 exigen medidas de seguridad proporcionales para datos personales. Guardar y comparar contraseñas sin hash incumple ese deber y, si se filtran, expone cuentas y datos de clientes. Es a la vez un defecto técnico y normativo.

**P: ¿La duplicación de 37 % en el backend es grave?**

R: No es bloqueante para certificar, pero afecta la mantenibilidad: patrones repetidos en routers aumentan el costo de corregir un defecto en varios lugares. Es deuda técnica que recomendamos reducir.

**P: ¿Cómo garantizan que las capturas de SonarQube son reales y no editadas?**

R: Porque provienen de un servidor SonarQube local desplegado en Docker, con los proyectos `atlas` y `atlas-frontend`, y el Anexo E incluye los comandos reproducibles del despliegue y del scan. Cualquiera puede repetir el análisis.

### 3.4. Sobre normativa y ética

**P: ¿Qué normas aplicaron y por qué esas?**

R: ISO/IEC 25010 para calidad de producto, ISO/IEC 27001 para controles de seguridad, OWASP Top 10 para riesgos web y las leyes 19.628 y 21.719 para protección de datos personales en Chile. Son las que exige el ERS y el contexto del caso.

**P: ¿Cómo se relaciona un defecto técnico con el cumplimiento normativo?**

R: Cada defecto crítico tiene una consecuencia normativa: el acceso sin token expone datos personales, las contraseñas sin hash incumplen el deber de seguridad y los errores 500 filtran información interna. Por eso el análisis normativo no es decorativo: se deriva de la evidencia.

**P: ¿Qué consideraciones éticas aplicaron?**

R: Reportar todos los hallazgos con evidencia, sin ocultarlos ni minimizarlos; probar solo en el entorno de laboratorio con datos ficticios; no difundir vulnerabilidades fuera del ámbito académico; y no modificar el código para ocultar problemas.

### 3.5. Sobre el dictamen y el cierre

**P: ¿Qué corregiría primero y por qué?**

R: Primero la autenticación global (BUG-001) y el hash de contraseñas (BUG-002), porque son las que exponen datos personales y permiten comprometer todo el sistema. Luego la validación de archivos (BUG-003) y los secretos en código (BUG-004 y BUG-011). Después los graves y, al final, la deuda técnica.

**P: ¿El sistema es funcional o no sirve?**

R: Es funcional en su base: 10 de 12 casos aprobados. El problema no es que no funcione, sino que no es seguro para producción. Un sistema puede ser funcional y aun así no ser certificable.

**P: ¿Qué evidencia respalda el dictamen?**

R: Los resultados de CP-010 y CP-011, las 7 pruebas de seguridad con resultado inseguro, los 4 incidentes CRITICO, la vulnerabilidad BLOCKER de SonarQube y el Security Rating E. Todo está en las secciones 5 y 6 del informe y en los anexos.

**P: Si tuvieran que repetir el trabajo, ¿qué cambiarían?**

R: Incorporaríamos OWASP ZAP y pruebas de carga con JMeter, automatizaríamos los casos de regresión y ejecutaríamos el análisis estático desde el primer día, para que las correcciones se validen en el mismo ciclo.

## 4. Frases de rescate y recomendaciones

- Si no recuerdas un dato exacto: "Ese dato específico está en el informe; lo que puedo afirmar con certeza es…" y responde con el hallazgo o la conclusión.
- Si la pregunta excede el alcance: "No lo medimos en este ciclo; lo dejo declarado como limitación y trabajo futuro".
- Si te piden demostrar: abre el informe (sección 5 y Anexo E), muestra las capturas y, si hay entorno, ejecuta `test_atlas.sh`.
- Ritmo: ensaya dos veces con cronómetro; si te pasas de 10 minutos, recorta las diapositivas 5 y 11, que son de apoyo.
- No leas las notas: úsalas como guía. Mantén contacto visual y cierra cada bloque con una idea fuerza.
- Ten a mano: informe EP3, Anexo E, presentación y este guion.

## 5. Checklist de entrega en AVA

- Informe EP3 en Word o PDF, con evidencias.
- Presentación `EP3_Presentacion_Atlas.pptx` (con notas del orador).
- Material de apoyo: este guion y el Anexo E.
- Verificar plazo de entrega y subir antes del cierre.

*Documento de apoyo para la defensa individual de la Evaluación Parcial N.º 4 (EP4). Complementa el informe EP3 y la presentación `EP3_Presentacion_Atlas.pptx`.*